



جامعة الأمير محمد بن فهد
PRINCE MOHAMMAD BIN FAHD UNIVERSITY

College of Computer Engineering and Science
Secure Software Engineering — Spring 2025–2026

Online Secure Student Information System (OSSIS)

Course Project Report

Prepared by:

Lama Alshehri - 202201859 , Mashael Alkadi - 202200507

Instructor: Dr. Maen Mohammad Mustafa Hammad

1. Introduction

1.1 General Description

The Online Secure Student Information System (OSSIS) is a web-based application for the secure management of student academic records. The system supports two types of users. Students have read only access to their own grades. Instructors are authorized to post and update grades for the courses they teach. All system interactions are performed through a browser-based interface supported by a secure server-side architecture and relational database.

Security is a critical concern at all stages of development of modern software systems, which are increasingly complex and interconnected. OSSIS is designed with security "built in" from the beginning, so that the confidentiality, integrity and availability of student data is protected throughout the life of the system.

1.2 Project Scope

The scope of OSSIS includes the design, analysis and implementation of a secure grade management platform. The system supports the following six functional operations specified in the project requirements:

Create Account - Register new student and Instructor accounts.

Update Account — Authenticated users can update existing account data.

Login – Secure login for students and instructors.

View Grades – Students can view their own grade records only.

Post Grades — Instructors submit grade records for students in their courses.

Update Grades Instructors update previously submitted grade records.

1.3 Objectives

The primary objectives of this project are the following:

1. To develop secure software system by implementing security principles in Software Development Life Cycle(SDLC).
2. To perform an entire security requirements analysis, including asset, threat and risk identification.
3. To develop the complete threat model using the STRIDE methodology of Microsoft and to apply appropriate security design principles.
4. To write defensive code for the login process, especially input validation and SQL injection prevention.
5. Define and document security test cases for all identified threats and all system functions.

2. Security Requirements Analysis

2.1 Functional and Non-Functional Requirements

Functional Requirements

The functional requirements specify the core capabilities the system must provide to its users. Each requirement maps directly to a system operation and an actor:

- FR-01: A new user (student or instructor) shall be able to register an account by giving a unique username and password.
- FR-02: An authenticated user shall be able to update their account information including contact details and password.
- FR-03: Registered users shall be able to log in using a valid username and password combination.
- FR-04: A verified student shall only be able to view grades linked to their own account.
- FR-05: An authenticated instructor shall be able to post grade records for students in their assigned courses.
- FR-06: An authenticated instructor will be authorized to edit grade records previously submitted by the students.

Non-Functional Security Requirements

Non-functional security requirements define the security features and limitations that a system must satisfy in addition to the functional behavior. These requirements are based upon industry standard secure coding practices and are vital to the protection of the system and its data:

- NFR-01: All user passwords must be stored as cryptographic hashes using the BCrypt algorithm so that no plaintext passwords are ever stored to the database.
- NFR-02: All data transmitted between the client and the server must be encrypted using Transport Layer Security (TLS 1.3).
- NFR-03: The system shall provide Role-Based Access Control (RBAC) to restrict students and instructors to only perform operations allowed by their respective role.
- NFR-04: All user inputs should be validated server side using a whitelist approach before any processing or database interaction is made.
- NFR-05: All requests to the database are required to be performed using parameterized statements. There must be no possibility of SQL injection attack.
- NFR-06: The system should lock a user account after three unsuccessful login attempts in a row. An administrator must be contacted to unlock the account.

- NFR-07: After 30 minutes of inactivity, an authenticated session will time out automatically.
- NFR-08: The audit log should keep a record of all login attempts (successful or not), including the time, date, username, IP address, and whether it was successful or not.

2.2 Security Requirements

Security requirements are derived from the functional and non-functional requirements. These are specific controls that the system need to implement to resist attack. The required security requirements for OSSIS are grouped into three categories:

- **Authentication and Authorization:** The system shall verify the identity of each user before providing access. Any authentication must be done by comparing BCrypt hashed passwords. Access to any system resource must be restricted by a server-side role check. The server shall not trust any role information provided by a client;
- **Data Protection** Student grade data and personal credentials are sensitive assets. Every password should be salted and hashed. All data in transit should be encrypted. All access to the database must be done through parameterized queries to prevent injection attacks. Do not expose sensitive information in error messages, logs, or HTTP responses unless it is absolutely required.
- **Resilience and Auditability** The system needs to be resistant to brute-force login attempts, so account lockout enforcement must be used. All authentication events shall be recorded in a tamper-evident audit trail. Session management must be resistant to fixation and hijacking attacks. Implement strict cookie security flags and regenerate sessions upon login.

2.3 Asset Identification

An asset is anything of value to the organization that must be protected, such as a resource, data item, or component. The following assets have been identified for the OSSIS:

- **User Credentials :** Usernames and BCrypt-hashed passwords in the user database. Compromise of this asset may allow unauthorized access to all system functions.
- **Grade Records :**Academic grades associated with student identities and course identifiers. These records are highly sensitive and must be safeguarded against unauthorized disclosure or alteration.
- **Personal Information ;** Names, university IDs and contact information of registered students and instructors.
- **Session Tokens :** Authentication tokens stored in cookies that identify active sessions of users. An attacker may be able to impersonate a legitimate user by stealing a session token.

- Database Server : The backend relational database that stores all data for the system. It is the single most important point of failure for data confidentiality and integrity.
- Audit Logs : Record of all authentication and grade change events. These should be protected against tampering to ensure non-repudiation.

2.4 Threat Identification

Threats are identified using the Microsoft STRIDE model that systematically addresses six categories of threats: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. The listed ten threats have been identified for OSSIS:

1. T-01 Spoofing: Attacker impersonates a legitimate user by obtaining or guessing user credentials and achieves unauthorized access to the system.
2. T-02 Tampering: An attacker makes forged HTTP requests to modify the grade records stored in the database.
3. T-03 Denial: An instructor modifies a grade record and subsequently denies the modification, exploiting the lack of a reliable audit trail.
4. T-04 Information Disclosure: A student manipulates request parameters (e.g., URL query strings) to retrieve another student's grade records.
5. T-05 Denial of Service: An attacker uses an automated brute-force attack to repeatedly submit invalid login credentials, potentially depleting system resources or locking out legitimate users.
6. T-06 Elevation of Privilege: A student exploits a missing or weakly enforced authorization check to access instructor-only features, such as grade posting.
7. T-07 SQL Injection: An attacker inserts a piece of malicious SQL code into a login or search field, tricking the database into executing commands to bypass authentication or to steal sensitive data.
8. T-08 Cross-Site Scripting (XSS): Malicious instructor inserts javascript code into a grade notes field that is run on other users' browsers.
9. T-09 Session hijacking: An attacker captures or predicts a session token to impersonate an authenticated user.
10. T-10 Broken Access Control: An unauthenticated or unauthorized user accesses a restricted page directly by manipulating the URL and bypassing the login requirement.

2.5 Risk Assessment and Management

Once a threat is identified, it is evaluated using a risk matrix, in which the Likelihood and Impact are rated using a three-point scale: 1 (Low), 2 (Medium), and 3 (High). The two values

are multiplied to produce the Risk Score, and threats are ranked accordingly. Table 1 summarizes the results.

Table 1: Risk Assessment Matrix

Threat ID	Threat	Likelihood	Impact	Risk Score	Priority	Mitigation
T-07	SQL Injection	3	3	9	Critical	Parameterized queries (PreparedStatement)
T-01	Credential Spoofing	3	3	9	Critical	BCrypt hashing; lockout after 3 failures
T-04	Information Disclosure	3	3	9	Critical	Server-side RBAC; user ID ownership check
T-06	Privilege Escalation	2	3	6	High	Role check enforced on every endpoint
T-09	Session Hijacking	2	3	6	High	TLS; Secure/HttpOnly cookies; session regeneration
T-10	Broken Access Control	2	3	6	High	Authentication filter on all protected routes
T-05	Denial of Service	2	2	4	Medium	Account lockout after 3 consecutive failures
T-08	Cross-Site Scripting	2	2	4	Medium	Output encoding; Content Security Policy header
T-02	Data Tampering	1	3	3	Medium	Server-side validation; DB integrity constraints
T-03	Repudiation	1	2	2	Low	Append-only audit log for all grade changes

2.6 Misuse Cases

Misuse cases describe scenarios where malicious actors intentionally violate system functionality. They are written from the attacker's perspective, complementing the threat analysis and guiding the design of specific counter-measures.

The most important misuse case is SQL Injection via the Login Form. The attacker injects a malicious string such as ' OR '1'='1'-- in the username or password field, exploiting insecurely constructed queries to bypass authentication and gain access to the system without any valid login credentials. The countermeasure is to always use parameterized PreparedStatement queries. This way, user input is never interpreted as SQL syntax.

Another important misuse case is Brute Force Login. An automated script is used to submit hundreds of password guesses for a known username. If you don't have a rate-limiting or lockout mechanism, the attacker will eventually find the correct password. To prevent this, the account is locked after three consecutive failed attempts.

Another misuse case is Unauthorized Grade Access where a student modifies the studentId parameter in a URL or API request to obtain another student's grade records. This is prevented by checking on the server that the requested student ID matches the ID of the currently authenticated session user.

Other misuse cases are Session Fixation where an attacker forces a known session identifier onto a victim before login, Stored XSS via grade note fields where a malicious instructor injects script tags that run in other users' browsers, and Direct URL Access to restricted administrative pages by unauthenticated users.

3. Securing Design

3.1 System Architecture and Components

OSSIS is designed as a three-tier web application, consisting of a presentation tier, an application tier and a data tier. This separation of concerns enables security controls to be independently applied to each layer, in accordance with the principle of defense in depth.

The Presentation Tier is the web browser client that the students and the instructors use to operate the system. The communication between the client and server is done exclusively through HTTPS. Each response comes with a Content Security Policy (CSP) header to restrict the sources from which scripts can be executed, reducing the chances of cross-site scripting attacks.

The Application Tier is implemented as Java Servlet based web application. This tier contains business logic, such as authenticating users, managing sessions, enforcing authorizations, validating inputs and auditing logs. An authorization filter intercepts every incoming request and checks if the requesting user has the appropriate role before passing the request to the appropriate handler. The filter represents the main enforcement point of Role-Based Access Control.

Data Tier: MySQL relational database to store user credentials, grade records, and audit logs. The application connects to the database using a dedicated service account with only the minimum privileges required (SELECT, INSERT, and UPDATE on the specific tables it requires). No privileges are assigned (DROP, CREATE, and GRANT). All database queries are sent from the application tier and are performed only through PreparedStatement objects, thus avoiding SQL injection at the architectural level.

The application tier contains the following key components: an Authentication Module, which handles credential verification, failed-attempt tracking, and session lifecycle; a Grade Management Module, which ensures that only instructors may post or update grades, and that students may only retrieve their own records; and an Audit Logger, which creates a structured record in the audit log for every authentication event and every grade modification.

3.2 Threat Modeling — Microsoft STRIDE Approach

The Microsoft threat modeling process was applied to OSSIS through the following five steps as defined in the SDL threat modeling methodology:

Step 1 – Identify Security Objectives. The primary objectives are to ensure the privacy of student grade information and credentials, protect the integrity of all grade records, and maintain the system accessible to authorized users. These goals directly map to the CIA triad and form the basis of mitigation assessment.

Step 2 – Develop Architecture Overview. The complete system components and their interactions were described in a Data Flow Diagram (DFD). The diagram identifies two external entities (Student Browser and Instructor Browser), three processes (Authentication, Grade Management, and Audit Logging), two data stores (User and Grades Database and Audit Log Database) and the trust boundaries between them. TLS enforces the trust boundary between

browser and web server, and network-level access control and parameterized queries enforce the trust boundary between application server and database.

Step 3 — Decompose the System. Each system component was decomposed to find entry points, exit points and data flows. Entry points are the login form (username and password entry fields), the account create form, the grade submission API endpoint and the grade view API endpoint. URL parameters are a secondary entry point and need to be validated server-side.

Step 4 – Identify Threats using STRIDE. Each component was evaluated against all six STRIDE categories. The login form is vulnerable to Credential Theft (T-01) via Spoofing and Malformed Input (T-07) via SQL Injection. Grade view endpoint is vulnerable to Information Disclosure (T-04) via parameter tampering and Broken Access Control (T-10). Tampering (T-02) and Elevation of Privilege (T-06) The grade management endpoint is vulnerable. Session Hijacking (T-09) is possible on the session management layer. Due to the lack of an audit log, there is a risk of Repudiation (T-03). Repeated failed login attempts are a Denial of Service vector (T-05) This process produced all ten threats identified in section 2.4.

Step 5 - Define Mitigations A corresponding security control was assigned for each identified threat. Spoofing is mitigated by BCrypt password hashing and account lockout. Use parameterized queries to prevent SQL Injection. Mitigation of Information Disclosure and Broken Access Control by server-side RBAC and user id ownership checks. TLS, Secure and Http to prevent Session Hijacking Only cookie flags and session regeneration on login. Account is Locked after three unsuccessful attempts to prevent Denial of Service. XSS is prevented using output encoding and CSP. The append-only audit log minimizes repudiation.

3.3 Security Design Principles

Principle of Least Privilege

In OSSIS, every user, process and database account is granted the minimum level of access required to do what it does. Students have read-only access to their own grade records. Instructors can only post and update grades for classes to which they are assigned. The database service account does not have any destructive privileges. This principle limits the blast radius of a successful attack by preventing a compromised account from accessing resources or inflicting damage outside the scope assigned to that account.

Password Security and Encryption

Passwords are never stored in plaintext and never stored in reversibly encrypted form . The system employs the BCrypt adaptive hashing function, which incorporates a random salt and a configurable cost factor, making precomputed dictionary and rainbow table attacks impractical. All network traffic is encrypted using TLS 1.3 to protect credentials and session tokens in transit. Any additional personally identifiable fields are encrypted at the column level with AES-256 where warranted.

Database Security

All of the data access layer uses parameterized queries with Java PreparedStatement objects. This design ensures that user input is always treated as a literal , and can never be confused for SQL syntax . This eliminates SQL injection vulnerabilities for good . Server-side environmental variables store database credentials, never hard-coded into source code.

Access Control

Role-Based Access Control is performed for each protected endpoint at the application tier. Central authorization filter inspects the role attribute stored in the server-side session before routing any request to a business-logic handler. The role is stored server-side so it cannot be manipulated by the client. Each grade retrieval request is also authenticated to ensure that the user's ID matches the ID of the owner of the resource.

Website Security and Session Management

Session tokens are stored in cookies with the Secure flag to prevent them being transmitted over unencrypted connections, the HttpOnly flag to prevent the cookie value being accessed by Javascript and the SameSite=Strict attribute to prevent cross-site request forgery. Sessions automatically expire after 30 minutes of inactivity. During a successful login existing session is invalidated and new session is created so that session fixation attacks are prevented.

Prevention Methods

In addition to the controls mentioned above, the following preventive mechanisms are implemented: All state-changing form submissions contain CSRF tokens; a Content Security Policy header restricts script sources; all user-supplied output is encoded before being rendered in HTML to prevent stored and reflected XSS; error messages returned to users are always generic, containing no information about whether a username exists or why authentication failed; and all login attempts are logged to a secure, append-only audit trail.

4. Secure Implementation

4.1 Secure Coding Guidelines

Secure coding guidelines are the set of practices and rules that developers must follow during the implementation phase to prevent the introduction of vulnerabilities. The following rules apply to the OSSIS development:

1. **Parameterized Queries Only** All database interactions must be done with `PreparedStatement` with parameters (placeholders) No dynamic SQL string construction using user supplied input, for any reason whatsoever.
2. **BCrypt Password Hashing:** Passwords should only be stored after being hashed using BCrypt with a cost factor of at least 12. Passwords in clear text must never be stored on disk, logged or otherwise transmitted beyond the initial HTTPS request.
3. **Server-Side Input Validation:** All input received from a client should be validated on the server (whitelist approach) before processing. Validation should validate the data type, length and format of the data against a defined pattern. Client-side validation by itself is not permitted.
4. **Least-Privilege Role Enforcement** Each privileged operation API endpoint must verify the authenticated user's role from the server-side session before execution. Do not trust any role information provided by the client.
5. **Session Fixation Prevention:** After a successful login the old session must be invalidated and a new session must be created. The new session ID must be generated by the application server's cryptographically secure random session manager.
6. **Secure Cookie Attributes:** Ensure all session cookies are issued with the Secure, HttpOnly, and SameSite=Strict attributes configured.
7. **Generic Authentication Error Messages:** Login responses should return the same message whether the username is unknown or the password is incorrect to prevent user enumeration.
8. **Account Lockout:** Account will be marked as locked in the database after three failed login attempts for a particular account and all subsequent login attempts will be refused until the lock is reset by an administrator.
9. **Output Encoding:** All user input data in an HTML document must be HTML encoded prior to output in order to prevent a cross-site scripting attack.
10. **No Hardcoded Credentials:** Secrets such as database connection strings, passwords, and API keys should be read from environment variables or a secrets manager and should never be in source code or version control.

11. **Defensive Error Handling:** Wrap all database and IO operations with try-catch blocks. Don't give the client stack traces and internal error details. Show a generic server error message.
12. **Audit Logging:** Record all authentication events and grade changes to an append-only audit log. The logged fields must include: the event type, username, source IP address, timestamp and outcome. Never write down passwords and session tokens.

4.2 Defensive Code for the Login Process

The implementation of the login is in the attached file `LoginService.java`. The code mitigates two main implementation level threats identified in the security analysis, for example improper input handling and SQL injection. The defense mechanisms used are described below.

Input Validation: The submitted username and password are validated for null or blank values, a maximum length of fifty characters, and a whitelist regular expression that allows only alphanumeric characters, underscores, and dots before any database interaction takes place. If the input does not pass these checks, then the request is immediately rejected with a generic error message and no database query is issued.

SQL Injection Prevention: The query uses a `PreparedStatement` with a single positional parameter placeholder. The username string is bound to this parameter with the `setString` method, which means that it will be sent to the database engine as a literal string value and none of it can be interpreted as SQL syntax. The insecure alternative of directly concatenating user input into a query string is never used.

Password Verification The submitted password is compared to the stored hash using the `BCrypt.checkpw` method which independently hashes the submitted value with the stored salt and performs a constant-time comparison. The hash can never be reversed or decrypted.

Account Lockout and Attempt Tracking For each login request the failed attempt count is pulled from the database. If the count is already three, the request is immediately rejected. If a password check fails, the counter in the database is incremented and when the new counter value hits three, the account is marked as locked. On a successful login the counter is reset to zero and the lock flag is cleared.

Session Management : After successful authentication the current `HttpSession` object calls `invalidate()` and the existing session is invalidated and a new session is created. This new session stores the user id, username and role that is authenticated. The session times out after 30 minutes of inactivity. A role-based redirect then directs the user to the correct dashboard.

5. Secure Testing

Security testing is the testing of the system against known threats and attack scenarios to verify that the security controls that were implemented during the design and coding phases work as expected. The test cases presented in this section are based on the threat analysis in Section 2 and consist of threat-specific and function-specific test cases.

5.1 Security Test Cases per Identified Threat

For every one of the ten threats defined by the STRIDE threat model, a security test case is defined. Table 2 shows how the mitigations designed and deployed for each threat behave under adversarial conditions, validating its correct behavior.

Table 2: Security Test Cases per Identified Threat

Test ID	Threat	Input / Action	Expected Result
STC-01	T-01: Credential Spoofing	3 failed logins then correct password for same account.	Login denied; account locked; correct password rejected.
STC-02	T-02: Data Tampering	Student sends POST /api/grades with modified grade parameter.	HTTP 403 Forbidden; grade not updated; action logged.
STC-03	T-03: Repudiation	Instructor posts a grade; audit log is examined afterward.	Log entry contains instructor ID, course, grade, and timestamp.
STC-04	T-04: Information Disclosure	Student changes studentId in URL to another student's ID.	HTTP 403; no grade data returned for the other student.
STC-05	T-05: Denial of Service	Automated script submits 10 login attempts with wrong password.	Account locked after 3rd failure; subsequent attempts rejected.
STC-06	T-06: Privilege Escalation	Student calls POST /api/grades using their own session token.	HTTP 403 Forbidden; grade not saved; error logged.
STC-07	T-07: SQL Injection	Username field: ' OR '1'='1'-- Password: anything.	Login fails; PreparedStatement blocks injection; attempt logged.
STC-08	T-08: Cross-Site Scripting	Instructor submits <code><script>alert(1)</script></code> in a grade note field.	Tag stored and rendered as escaped text; no script executes.
STC-09	T-09: Session Hijacking	Previously captured session cookie replayed after user logs out.	Session invalid; HTTP 401; user redirected to login page.
STC-10	T-10: Broken Access Control	Unauthenticated browser navigates directly to /dashboard/instructor.	HTTP 302 redirect to login page; no dashboard content returned.

5.2 Security Test Cases per System Function

For each of the six system functions described in the project scope, two security test cases are provided. These twelve tests together verify that each function is secure under both attack conditions and boundary input scenarios as shown in Table 3 .

Table 3: Functional Security Test Cases

Test ID	Function	Type	Input / Action	Expected Result
FTC-01A	Create Account	SQL Injection	Username: admin'-- Password: Test@123	Registration rejected; validation error; no database record created.
FTC-01B	Create Account	Boundary	Username: 200-character string; Password: valid	Input rejected; error: username exceeds maximum length.
FTC-02A	Update Account	IDOR	PUT /account?userId=OTHER_ID with own valid session.	HTTP 403; only own account is editable; access denied logged.
FTC-02B	Update Account	Weak Password	New password: 12 (two characters)	Rejected; password policy error: minimum length and complexity not met.
FTC-03A	Login	SQL Injection	Username: admin Password: ' OR '1'='1	Login fails; query uses PreparedStatement; bypass not possible.
FTC-03B	Login	Account Lockout	3 wrong passwords submitted, then correct credentials.	Login denied after 3rd failure; account locked; correct password also rejected.
FTC-04A	View Grades	IDOR	GET /grades?studentId=DIFFERENT_ID with own student session.	HTTP 403; only own grades accessible; request denied.
FTC-04B	View Grades	Unauthenticated	GET /grades without any active session cookie.	HTTP 302 redirect to login page; no grade data returned.

Test ID	Function	Type	Input / Action	Expected Result
FTC-05A	Post Grades	Privilege	POST /api/grades with student-role session token.	HTTP 403 Forbidden; grade not saved; unauthorized action logged.
FTC-05B	Post Grades	XSS	Grade note field: 	Input sanitized on output; rendered as escaped text; no script executes.
FTC-06A	Update Grades	Cross-Role	Instructor updates a grade for a course they are not assigned to.	HTTP 403 Forbidden; grade unchanged; access denied event logged.
FTC-06B	Update Grades	Invalid Value	PUT /api/grades/101 with grade = -50	Validation error returned; grade record unchanged in database.